Security Assessment
Final Report

1Delta
Compound Integration

December 2025

Prepared for 1Delta DAO and Compound

Table of contents

Project Summary 3
Project Scope 3
Project Overview 3
Protocol Overview 3
Findings Summary 4
Severity Matrix 4
Detailed Findings 5
Medium Severity Issues 6
M-01 Permit operations vulnerable to frontrunning griefing attack 6
M-02 External protocol upgrade could drain approvals 8
Low Severity Issues 9
L-01 Ignores vToken boolean return; uses unsafe transfers 9
L-02 Oversized splits not rejected 10
Informational Issues 11
I-01. Missing lender ID validation in UniversalLending router 11
I-02. Amount masking inconsistency in withdraw function 12
I-03. Uses garbage when staticcall fails 13
I-04. “CompoundV2Lending” is Venus-only due to borrowBehalf 13
I-05. Misleading comment in assembly operation parsing 13
Disclaimer 15
About Certora 15

Project Summary

Project Scope

Project Name	Repository (link)	Initial Commit Hash	Platform
cap	1delta-DAO/contracts-delegation	bea581f	EVM

Project Overview

This document describes the verification of **1Delta Compound Integration** using manual code review. The work was undertaken from **December 10th, 2025** to **December 17th, 2025**.

The team performed a manual audit of the following Solidity contracts:

- `contracts/1delta/composer/BaseComposer.sol`
- `contracts/1delta/composer/swappers/BaseSwapper.sol`
- `contracts/1delta/composer/lending/UniversalLending.sol`
- `contracts/1delta/composer/lending/CompoundV2Lending.sol`
- `contracts/1delta/composer/lending/CompoundV3Lending.sol`
- `contracts/1delta/shared/permit/PermitUtils.sol`
- `contracts/1delta/composer/chains/ethereum/flashSwap/callbacks/BalancerV3Callback.sol`

During the manual audit, the Certora team discovered bugs in the Solidity contracts code, as listed on the following page.

Protocol Overview

1Delta is a decentralized margin aggregator service that allows users to directly build margin positions using established DEXes and lending protocols.

The audit focuses on 1Delta's integration with Compound v2 and v3.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	-	-
High	0	-	-
Medium	2	2	1
Low	2	2	1
Informational	5	5	4
Total	10	10	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Permit operations vulnerable to frontrunning griefing attack	Medium	Fixed
M-02	External protocol upgrade could drain approvals	Medium	Acknowledged
L-01	Ignores vToken boolean return; uses unsafe transfers	Low	Fixed
L-02	Oversized splits not rejected	Low	Fixed
I-01	Missing lender ID validation in UniversalLending router	Informational	Fixed
I-02	Amount masking inconsistency in withdraw function	Informational	Fixed
I-03	Uses garbage when staticcall fails	Informational	Acknowledged
I-04	“CompoundV2Lending” is Venus-only due to borrowBehalf	Informational	Fixed
I-05	Misleading comment in assembly operation parsing	Informational	Fixed

Medium Severity Issues

M-01 Permit operations vulnerable to frontrunning griefing attack		
Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: PermitUtils.sol#L117-L121	Status: Fixed	

Description: The protocol's permit operations can be griefed by attackers who frontrun transactions and execute permit signatures before the original user's transaction processes.

This attack pattern is well-known and documented in the following blog post:

https://www.trust-security.xyz/post/permission-denied

Attackers extract valid permit signatures from the mempool and execute them first. This increments the permit nonce and causes the original transaction to revert when the same permit signature is used again. Since permit failures cause the entire `deltaCompose()` transaction to revert, users lose gas costs and their intended operations fail completely.

Attack scenario:

1. User submits `deltaCompose()` with operations:

   ```
   [permit USDC][swap USDC→ETH][deposit ETH to Compound]
   ```

2. MEV bot extracts the permit signature from mempool and frontruns with the same permit signature at higher gas price
3. The bot's transaction executes first, permit succeeds, nonce increments
4. User's transaction executes but permit fails due to used nonce
5. As a result the entire `deltaCompose()` reverts, user gets no execution and loses gas

This griefing attack has moderate likelihood since it is a known MEV strategy with established tooling. The attack cost is minimal while providing guaranteed disruption to user transactions that rely on permits.

Recommendations: There are two potential mitigation approaches:

1. **Graceful `permit()` handling** - modify `_tryPermit()` to check allowances after permit failure. Only revert if the current allowance is insufficient for the intended operation.
2. **Optional revert flag** - add a boolean parameter to permit operations indicating whether permit failures should revert or continue. This gives users control over transaction behavior while maintaining backward compatibility for safety-critical operations.

Customer's response: We use pop(call(...)) in calling permits, this avoids reverts if a frontrunner extracts and executes the permit before being composed txn if the composer does not have the required allowance, it will revert in further operations.

PermitUtils.sol#L65, PermitUtils.sol#L87, PermitUtils.sol#L110

Fix review: Looks good.

M-02 External protocol upgrade could drain approvals

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: -	Status: Acknowledged	

Description: The composer calls external protocols. If any integrated protocol's governance is compromised and upgrades its contracts, it could make arbitrary calls back into the composer and spend lingering user approvals. In practice, the protocol's security is the intersection of all integrated protocols' governance security—one compromised integration can break the whole.

Recommendations: Prefer expiring approvals (e.g., Permit2 with short deadlines), avoid broad/persistent approvals, and constrain callable targets via allowlists/version pinning. There's limited on-chain mitigation; this is a systemic risk users must understand and manage (hence Medium).

Customer's response: Acknowledged

Fix review: This finding is intended to make the end user aware of the risks associated with the protocol.

Low Severity Issues

L-01 Ignores vToken boolean return; uses unsafe transfers		
Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: CompoundV2Lending.sol#L178	Status: fixed	

Description: `vToken` returns a boolean to signal success (they do not revert on failure), but the code does not check the return value. Likewise, transfers are done without safe wrappers. This can silently fail (e.g., token not transferred/repay not executed) yet the flow proceeds as if successful, leading to inconsistent accounting.

Recommendations: Wrap `vToken` interactions with return-value checks; revert on false. Use `safeTransfer/safeTransferFrom` (or equivalent).
Good practice is to consistently use `safeERC20` lib over the codebase to ensure failures are caught instead of ignored. (not only for `vTokens` but for all ERC20)

Customer's response: **CompoundV2Lending.sol#L183-L199**

Fix review: Fix looks good.

L-02 Oversized splits not rejected

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: [BaseSwapper.sol#L178](#)	Status: Fixed	

Description: In `BaseSwapper` splits are decoded from a single 16-byte chunk, so at most 8 uint16 split ratios can be read. `splitsMaxIndex` comes from user calldata and controls how many splits the decoder expects. `splitsMaxIndex` is unbounded. If it's set > 7, the decoder still skips `2 * splitsMaxIndex` bytes even though only the first 8 entries are readable; anything beyond is zeros or misaligned calldata, which then gets interpreted as `(r,c)`, `nextToken`, `receiver`, and `dex` params. That can misroute swaps or cause reverts.

Recommendations: Enforce the limit:

```rust
if (splitsMaxIndex > 7) revert InvalidCalldata();
```

before decoding splits.

Customer's response: [BaseSwapper.sol#L275](#)

Fix review: Fix looks good.

Informational Issues

I-01. Missing lender ID validation in UniversalLending router

Description: The `_lendingOperations()` function extracts lender IDs from calldata without validating they fall within defined ranges, potentially causing undefined routing behavior.

The function routes operations based on range comparisons (`UP_TO_AAVE_V3 = 1000`, `UP_TO_AAVE_V2 = 2000`, etc.) but does not validate that lender IDs are within the valid range of 0-5999. Invalid lender IDs above 6000 (`UP_TO_SILO_V2`) will fall through all conditional checks and incorrectly route to SiloV2 operations, despite the naming convention indicating SiloV2 should only handle IDs below 6000.

Additionally, operations `DEPOSIT_LENDING_TOKEN` and `WITHDRAW_LENDING_TOKEN` completely ignore the lender ID parameter and always route to Morpho functions. Users specifying different lender IDs for these operations will have their transactions unexpectedly routed to Morpho regardless of their intent.

This could cause transactions to interact with wrong lending protocols or fail unexpectedly, resulting in wasted gas and user confusion.

Recommendation: Add explicit validation for lender ID ranges. This ensures malformed lender IDs are rejected early with a clear error rather than causing unexpected routing behavior.

Customer's response:

UniversalLending.sol#L74, UniversalLending.sol#L110, UniversalLending.sol#L92, UniversalLending.sol#L143

Fix review: Fixed, Looks good.

I-02. Amount masking inconsistency in withdraw function

Description: The `CompoundV3Lending#_withdrawFromCompoundV3()` and `CompoundV3Lending#_borrowFromCompoundV3()` functionc use inconsistent amount masking that does not align with the protocol's intended `uint112` amount representation.

The protocol uses `uint112` (14 bytes) to represent external protocol amounts with the highest 2 bytes reserved for flags such as shares mode (Morpho) or collateral mode (SiloV2). However, the withdraw function extracts amounts using `UINT120_MASK` (15 bytes) while checking for the maximum value using a `type(uint112).max` constant. This creates inconsistency between the extraction logic and the intended data format.

For the current Compound V3 integration, this has no functional impact since Compound operations do not use flag bits in amount data. However, the masking should use `UINT112_MASK` to maintain consistency with the protocol's `uint112` amount representation used across other lending integrations.

Note: while outside the scope of this Compound-focused audit, this inconsistency would break "withdraw all" functionality in Morpho and SiloV2 integrations when flag bits are set as the comparison would fail due to the included flag data.

Recommendations: Use `UINT112_MASK` for consistent amount handling:

```
None
let amount := and(UINT112_MASK, amountData)  // Use uint112 consistently
if eq(amount, 0xffffffffffffffffffffffffffff) {
```

Customer's response: [CompoundV3Lending.sol#L212]

Fix review: Fixed, Looks good.

I-03. Uses garbage when staticcall fails

Description: Across the codebase, staticcall return values are often ignored (e.g., `CompoundV3Lending._withdrawFromCompoundV3` at CompoundV3Lending.sol#L51-L62). That's bad practice: on failure the buffer is never written, and you end up using garbage data. Saving a few gas is not worth the risk of arbitrary, unintended behavior. Return values should be checked; revert early on failure.

Recommendations: Check the `staticcall` return value and revert on failure.

Customer's response: Acknowledged

Fix review: -

I-04. "CompoundV2Lending" is Venus-only due to borrowBehalf

Description: The implementation calls `borrowBehalf(address,uint256)`, which exists on Venus vTokens but not on Compound V2 cTokens (which only have `borrow(uint256)` for `msg.sender`). As written, borrowing will fail on vanilla Compound V2, so the module is not Compound-compatible despite its name.

Recommendations: Rename the file/contract to `VenusLending` to avoid misleading users and integrators.

Customer's response: Fixed, we added a comment on the CompoundV2Lending file as the venus-specific parts are 3 sub-functions only (borrowBehalf, mintBehalf & redeemBehalf).

Fix review: OK.

I-05. Misleading comment in assembly operation parsing

Description: The assembly code incorrectly comments `shr(248, calldataload(currentOffset))` as extracting the "last byte" when it actually extracts the first byte.

The operation `shr(248, value)` shifts right by 248 bits, leaving only the leftmost 8 bits (first byte). This is standard for extracting the most significant byte from a `bytes32` value.

While functionally correct, misleading comments in assembly code create risks as future developers might rely on the incorrect comment when modifying the operation parsing logic, potentially introducing bugs.

Recommendations: Update the comment to reflect actual behavior:

```rust
operation := shr(248, calldataload(currentOffset)) // first byte
```

Customer's response: Fixed

Fix review: OK.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

The review was conducted over a limited timeframe and may not have uncovered all relevant issues or vulnerabilities.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.